

初始化可能失败的单例模式，author：码农Mark

问题描述

在多线程环境下，需要实现一个线程安全的单例模式，该类包含一个初始化函数 `init()`，该函数可能非常耗时，且**必须成功初始化才能返回单例对象，否则应返回 `nullptr`**。需要解决的关键问题包括：

- 确保单例对象只被初始化一次
- 处理初始化过程中可能发生的失败情况
- 保证多线程环境下的线程安全性

单例模式的基本要求

实现一个符合要求的单例模式需要满足以下基本要求：

- **私有构造函数**：防止外部直接创建对象
- **私有析构函数**：防止外部直接销毁对象
- **禁用拷贝和移动操作**：防止通过拷贝创建多个实例
- **全局唯一访问点**：提供 `GetInstance()` 等静态方法获取单例
- **线程安全**：多线程环境下，保证单例的初始化只执行一次
- **初始化失败处理**：确保初始化失败时返回 `nullptr`，不留下部分初始化状态

双重检测锁实现

```
1  #pragma once
2
3  #include <atomic>
4  #include <mutex>
5
6  #include <stdexcept>
7  #include <iostream>
8
9  class Singleton_V1 {
10 public:
11     static Singleton_V1* GetInstance() {
12         Singleton_V1* inst = instance_.load(std::memory_order_acquire);
13         if (inst == nullptr) {
14             std::lock_guard<std::mutex> lock(mutex_);
15             inst = instance_.load(std::memory_order_relaxed);
16             if (inst == nullptr) {
17                 try {
18                     inst = new Singleton_V1();
19                     /*
20                      编译器可能优化：
21                      1. 先分配内存
22                      2. 立即将指针存储到 instance_
23                      3. 然后才调用构造函数
24                      inst = static_cast<Singleton_V1*>(operator
25 new(sizeof(Singleton_V1)));
26                      instance_.store(inst, std::memory_order_release);
27                      new (inst) Singleton_V1();
28                      */
29                 } catch (...) {
30                     return nullptr;
31                 }
32             }
33         }
34         return inst;
35     }
36 }
```

```

27         */
28         // std::atomic_thread_fence(std::memory_order_release);
29         // instance_.store(inst, std::memory_order_release);
30         instance_.store(inst, std::memory_order_release);
31         std::atexit(&Destroy);
32     } catch(const std::exception& e) {
33         // TODO:记录失败日志
34         return nullptr;
35     }
36 }
37 }
38 return inst;
39 }
40
41 void dosomething() {
42     std::cout << "Singleton_V1 doing something." << std::endl;
43 }
44
45 private:
46     // static inline : c++ 17
47     static inline std::atomic<Singleton_V1*> instance_ = {nullptr};
48     static inline std::mutex mutex_;
49
50     bool Init() {
51         // 初始化操作 (可能耗时)
52         return true; // 实际中根据需求实现
53     }
54
55     Singleton_V1() {
56         if (!Init()) {
57             throw std::runtime_error("初始化失败");
58         }
59     }
60
61     static void Destroy() {
62         Singleton_V1 *inst = instance_.exchange(nullptr);
63         delete inst;
64     }
65
66     ~Singleton_V1() {
67         std::cout << "Singleton_V1 destructed." << std::endl;
68     }
69
70     Singleton_V1(const Singleton_V1 &) = delete;
71     Singleton_V1& operator=(const Singleton_V1&) = delete;
72
73     Singleton_V1(Singleton_V1&&) = delete;
74     Singleton_V1& operator=(Singleton_V1&&) = delete;
75 };

```

设计要点

- **双重检查锁模式**：先检查原子指针，再加锁确保初始化过程只执行一次；
- **异常安全**：
 - 内存分配失败，直接返回 `nullptr`
 - 初始化失败，构造函数中抛出异常，自动释放内存，此时 `instance_` 没有被设置，接着返回 `nullptr`；

静态局部变量实现

```
1
2 #pragma once
3 #include <stdexcept>
4
5 #include <iostream>
6
7 class Singleton_V2 {
8 public:
9     static Singleton_V2* GetInstance() {
10         try {
11             static Singleton_V2 instance;
12             return &instance;
13         } catch (const std::exception& e) {
14             // TODO:记录失败日志
15             return nullptr;
16         }
17     }
18
19     void dosomething() {
20         std::cout << "Singleton_V2 doing something." << std::endl;
21     }
22
23 private:
24     bool Init() {
25         // 初始化操作 (可能耗时)
26         return true; // 实际中根据需求实现
27     }
28
29     Singleton_V2() {
30         if (!Init()) {
31             throw std::runtime_error("初始化失败");
32         }
33     }
34     ~Singleton_V2() {
35         std::cout << "Singleton_V2 destructed." << std::endl;
36     }
37
38     Singleton_V2(const Singleton_V2 &) = delete;
39     Singleton_V2& operator=(const Singleton_V2&) = delete;
40
41     Singleton_V2(Singleton_V2&&) = delete;
42     Singleton_V2& operator=(Singleton_V2&&) = delete;
43 };
```

设计要点

- **线程安全**：利用 `C++11` 的线程安全静态局部变量特性
- **自动内存管理**：利用 `RAII` 机制，程序退出自动释放资源
- **异常处理**：如果初始化失败，构造函数会抛出异常，导致 `GetInstance()` 返回 `nullptr`，此时 `instance` 没有成功创建，下次仍会继续初始化；